Université Côte d'Azur

Filtering event-based data using saliency maps

Development Project

Year 2026

Students:

Giulia Raffaella Vitale – M2 IF

Eya Ben Moulehem – M2 IF

Ala Eddine Ben Mimoun – M2 IF

Supervisor:

Jean Martinet

Romaric Mazna

Contents

1 Introduction **3**
1.1 Context . . . 3
1.2 Problem Statement . . . 3
1.3 Objectives . . . 4
1.4 Scope . . . 5

2 Proposed Solution **6**
2.1 Possible Solutions . . . 6
2.2 Retained Solutions . . . 7

3 State of the Art **9**
3.1 Existing Solutions . . . 9
3.2 Positioning . . . 10

4 Work Performed **12**
4.1 Data Pipeline and Methodology . . . 12
4.2 Event Data and Datasets . . . 12
4.3 Filtering Techniques Implemented . . . 13
4.3.1 Hyperparameter Tuning and Optimization . . . 14
4.3.2 The Event Reduction Ratio (ERR) . . . 14
4.3.3 OMS: Motion-Based Saliency Filtering . . . 15
4.3.4 Attention-Based Saliency Filtering . . . 16
4.3.5 Adaptive Elbow OMS Filtering . . . 17
4.3.6 Goal-Oriented OMS Filtering . . . 18
4.3.7 Mean and Standard Deviation Thresholding . . . 20
4.3.8 Global Saliency-Based Cropping . . . 21
4.3.9 Denoising . . . 23
4.3.10 Random Crop Filtering . . . 23
4.4 Downstream Task: Classification Setup . . . 25
4.4.1 Data Representation and Converters . . . 25
4.4.2 Model Architecture: DVSGestureCNN . . . 25
4.4.3 Training Methodology and Workflow . . . 26
4.4.4 Hyperparameters Summary . . . 26
4.5 Experimental Environment . . . 26

5 Results **28**
5.1 Experimental Setup 28
5.2 Quantitative Results 28
5.3 Accuracy vs. Data Reduction 29
5.4 Computational Efficiency 30
5.5 Comparative Accuracy Analysis 30
5.6 Performance Observations 31
5.7 Trade-Off Analysis 31
5.8 Limitations 32

6 Conclusions **33**
6.1 Future Work 34

7 References **35**

1 Introduction

1.1 Context

Event-Based Data

Event-based cameras, also known as neuromorphic cameras, differ fundamentally from conventional frame-based cameras. Instead of capturing full images at fixed time intervals, they asynchronously record changes in brightness at each pixel. The output is a stream of events defined by spatial location, timestamp, and polarity. This sensing paradigm offers several advantages:

- Very low latency
- High temporal resolution
- Reduced motion blur
- Lower data redundancy in dynamic scenes

These properties make event-based vision particularly suitable for applications such as robotics, autonomous driving, and gesture recognition. However, event streams also present challenges. They often contain background activity noise, redundant events, and irrelevant motion, which may negatively affect downstream tasks

Saliency Maps

A saliency map is a representation that highlights the most informative or relevant regions of data. In the context of event-based vision, saliency maps aim to identify meaningful event patterns while suppressing noise and less informative activity. By focusing on salient events, filtering techniques guided by saliency maps can potentially improve data quality before it is processed by downstream models.

1.2 Problem Statement

Event-based vision systems generate large streams of asynchronous events that may contain noise, redundancy, and background activity. To address this, various filtering techniques are commonly applied before performing downstream tasks such as classification.

Filtering aims to remove irrelevant events and highlight meaningful patterns, often using saliency-based approaches. While these techniques reduce the number

of events and potentially improve data quality, their true impact on downstream performance remains unclear.

In particular, an important question arises:

Does filtering event-based data improve classification performance, or can raw event data be equally effective, or even superior, in certain cases?

Furthermore, aggressive filtering may remove useful information along with noise, potentially degrading model performance.

Therefore, the central challenge of this project is to systematically evaluate whether saliency-based filtering techniques provide measurable benefits for downstream classification compared to using raw event streams.

1.3 Objectives

The main objective of this project is to investigate the impact of saliency-based filtering techniques on downstream performance in event-based vision. To achieve this goal, the project pursues the following specific objectives:

- Implement and optimize multiple event-based filtering techniques, including OMS, Attention mechanisms, and OMS combined with different saliency-based masks.
- Design a controlled experimental framework where only the filtering stage varies, ensuring fair comparison with raw event data.
- Evaluate and compare the classification performance obtained from raw and filtered event streams.
- Analyze the trade-off between event reduction, computational efficiency, and classification accuracy.
- Identify which filtering strategies, if any, provide measurable improvements in accuracy and robustness over raw event data.

Through these objectives, the project aims to provide a systematic and quantitative assessment of whether filtering genuinely benefits downstream classification tasks in event-based vision.

1.4 Scope

- The project focuses on evaluating filtering techniques for event-based data using OMS, Attention, and Mask-based methods.
- Two datasets where considered to evaluate the filtering techniques, DVS Gesture dataset and DSEC dataset.
- Downstream classification is performed only on the DVS Gesture dataset, while filtering evaluation (visual analysis, metrics) may also include other datasets like DSEC.
- Parameter tuning and comparison of filtering impact on classification accuracy and reliability are included.
- The project does not cover classification on datasets without labels suitable for downstream tasks.

2 Proposed Solution

To address the research problem, several possible approaches were considered for evaluating the impact of filtering on downstream classification performance.

2.1 Possible Solutions

The following solution space was explored:

- Using raw event data directly for the downstream classification task (baseline).
- Tonic built-in filtering techniques (four different methods).
  - Inter-Event Interval
  - Refractory Period Leaky
  - Integrate-and-Fire
  - Denoise
- Using saliency-based filtering techniques, with parameter tuning.
- Combining saliency filtering with different masking strategies (five mask variants), with parameter tuning. Different mask strategies were considered, such as:
  - Goal-Oriented Percentile
  - Adaptive Elbow
  - Mean/Standard Deviation
  - Global Saliency-Based Cropping
  - Combined Global + Instance-Specific Cropping
- Applying an attention mechanism directly on the raw dataset.

These approaches represent different philosophies: no filtering, classical filtering, heuristic saliency filtering, mask-enhanced saliency selection, and learning-based attention mechanisms.

2.2 Retained Solutions

After preliminary evaluation, the retained solutions were:

- Using raw event data directly (baseline).
- Applying one selected Tonic filtering method.
  - Denoise
- Applying saliency-based filtering techniques (with tuned parameters).
- Applying saliency combined with four selected mask strategies (with tuned parameters).
  - Goal-Oriented Percentile
  - Adaptive Elbow
  - Mean/Standard Deviation
  - Global Saliency-Based Cropping
- Applying an attention mechanism on the raw dataset.

The selection of the retained filtering techniques was primarily based on the Event Reduction Ratio (ERR). Filters that did not significantly reduce the number of events were not retained, as they did not contribute meaningfully to data simplification or noise reduction.

Based on the filtering results across both datasets, we made a selection of techniques to retain for further experiments. Filters that showed negligible impact those with ratios close to 0 or extremely low event reduction were discarded, as they do not meaningfully reduce data or contribute to improving downstream classification. The remaining techniques, which demonstrated substantial filtering ratios, were kept for detailed evaluation and comparison, ensuring that only effective and informative event-filtering methods are considered in subsequent analysis. The detailed ERR results for each filtering technique on both datasets (DSEC and DVS Gesture) are presented in the following section.

Filtering Technique	**Mechanism**	**Dataset**	**Ratio**
Tonic	Inter-Event Interval (IEI)	DSEC	0.00
		DVS Gesture	0.0417
	Refractory Period Leaky	Dsec	0.00
		DVS Gesture	0.0417
	Integrate-and-Fire (LIF)	DSEC	0.00
		DVS Gesture	0.00
	Denoise	Dsec	0.0352
		DVS Gesture	0.3246
Spatio-Temporal Saliency Extraction via OMS		Dsec	0.2414
		DVS	0.2136
Attention		Dsec	0.7362
		DVS	0.1464
OMS Map + Mask	Goal Oriented Percentile	Dsec	0.4433
		DVS Gesture	0.1208
	Adaptive elbow	Dsec	0.7343
		DVS Gesture	0.9658
	Mean/Standard Deviation	Dsec	0.3617
		DVS Gesture	0.1679
	Global Saliency-Based Cropping	Dsec	0.0652
		DVS Gesture	0.3160
	Combined Global + Instance-Specific Cropping	Dsec	0.0001
		DVS Gesture	0.3160

Table 1: Filtering Techniques and Performance Ratios across Datasets

3 State of the Art

Current research in event-based vision filtering and object detection proposes several methodologies to handle sensor noise, data redundancy, and computational constraints. Three prominent approaches are detailed below.

3.1 Existing Solutions

Wandering around: A bioinspired approach to visual attention through object motion sensitivity [2]

This approach addresses the challenge of detecting and focusing visual attention on relevant objects in dynamic scenes, mitigating disturbances from sensor noise and camera egomotion. The proposed system utilizes a biologically inspired visual attention mechanism combining Object Motion Sensitivity (OMS) with proto-object saliency and active attention mechanisms. It operates in real time on neuromorphic hardware using spiking neural networks (SNNs) without requiring offline learning.

- **Advantages**: The spiking OMS module eliminates approximately 85% of background events, substantially reducing the computational load. It achieves high motion segmentation performance (up to 82% Intersection over Union) and low latency (0.124 seconds), making it suitable for robotics.
- **Limitations**: The system's performance degrades when background motion exceeds object motion. It is restricted to selecting a single saliency maximum, preventing simultaneous detection of multiple objects. Additionally, its impact on downstream artificial intelligence object recognition models is not directly evaluated.

Object Detection for Embedded Systems Using Tiny Spiking Neural Networks: Filtering Noise Through Visual Attention [1]

This methodology targets the high computational and energy costs associated with running state-of-the-art deep neural networks on resource-constrained embedded systems. It proposes a lightweight, learning-free SNN architecture that uses bio-inspired visual attention to filter noise implicitly, bypassing the need for explicit refractory noise-filtering layers. The pipeline applies spatial funnelling to downscale the event stream, processes patches through a Region of Interest (ROI) layer using Leaky Integrate-and-Fire neurons, and implements lateral excitation, distance-dependent inhibition, and adaptive synaptic weights.

- **Advantages**: The network is extremely compact, utilizing only 192 neurons compared to nearly 5,000 in comparable detectors. It achieves competitive precision and recall, outperforms prior SNN methods in high-noise environments, and demonstrates high energy efficiency on SpiNNaker hardware.

- **Limitations**: Object detection relies on simple clustering, limiting the system to generating coarse bounding boxes without handling complex shapes. The system strictly performs detection without assigning semantic labels, and its evaluation is limited exclusively to pedestrian detection. The real-time efficiency claims remain closely tied to the specialized SpiNNaker hardware.

Neuromorphic foveation applied to semantic segmentation [3]

This solution tackles the computational load of processing full-resolution asynchronous event volumes by mimicking biological foveation. An SNN identifies salient Regions of Interest (ROIs) based on event density. Events within the fovea are processed at high spatial resolution, while peripheral events are processed at a lower resolution. The resulting combined foveated stream is then routed to an Ev-SegNet model for semantic segmentation.

- **Advantages**: The technique significantly reduces data volume, retaining an average of 30% of events while maintaining semantic segmentation performance close to full-resolution processing. It operates with low latency by relying on SNN dynamics asynchronously, reducing memory access and power consumption.

- **Limitations**: The current implementation is restricted to binary foveation (fovea versus periphery) without multi-level resolution hierarchies. Validation is constrained to semantic segmentation, and the system relies entirely on event density as a proxy for relevance, which may discard important data in low-contrast scenarios. Furthermore, the saliency SNN and the segmentation network lack joint, end-to-end optimization.

3.2 Positioning

While existing approaches demonstrate significant advancements in event-based noise reduction and saliency detection, they frequently exhibit a critical gap regarding integration with standard artificial intelligence perception pipelines. Bio-inspired models and neuromorphic foveation techniques excel at minimizing computational loads and highlighting relevant dynamic objects. However, their evaluation remains

predominantly confined to active visual attention, motion segmentation, or low-level bounding box generation.

Specifically, prior works do not directly assess the impact of these neuromorphic preprocessing and filtering steps on the performance of downstream object recognition and classification models. Furthermore, many current state-of-the-art systems are heavily dependent on specific neuromorphic hardware implementations, such as SpiNNaker or the Speck platform, limiting their immediate transferability to conventional software pipelines or standard convolutional neural networks.

Our work positions itself directly within this identified gap between neuromorphic preprocessing and full AI-based object recognition. Instead of solely optimizing a standalone neuromorphic filter, this project systematically investigates the core research question: what is the precise impact of saliency-based event filtering on downstream tasks? By implementing and comparing various spatial filtering strategies specifically global saliency-based cropping, instance-specific dynamic bounding boxes, and combined approaches we evaluate the direct trade-off between event data reduction and task performance. This methodology determines how effective event filtering can optimize data streams for conventional classification and detection frameworks without discarding critical semantic information.

4 Work Performed

This chapter details the practical implementation of the project, focusing on both the methodological process and the final analytical pipeline developed. The primary objective is to evaluate the viability of applying filtering techniques to event data prior to downstream processing. To achieve this, a complete experimental workflow was constructed to compare the performance of machine learning classification on raw, unfiltered event data against data reduced through various filtering methods. The following sections outline the architecture of the data pipeline, the specific filtering algorithms implemented, and the configuration of the classification tasks. A comparative analysis of the resulting accuracy and computational trade-offs will follow in the next chapter.

4.1 Data Pipeline and Methodology

The analytical framework was structured as an iterative pipeline designed to transition from raw event streams to evaluated classification results. The process began with the acquisition of the datasets followed by an analysis of potential filtering approaches. The selection criteria focused on methods that offered a balance between data reduction and the preservation of high-fidelity temporal information. The implementation phase started with the integration of the Object Motion Sensitivity (OMS) saliency map, as proposed in the work of D'Angelo et al. (2020) [2], which mimics biological retinal mechanisms to filter redundant background activity. Building upon this, various spatial and temporal masks were developed to be applied over the OMS output to further refine the event stream. In parallel, independent filtering benchmarks were established using standard algorithms, such as the *Denoise* filter provided by the Tonic framework, which utilizes a background activity filter based on spatiotemporal correlation. Finally, the filtered data was fed into a downstream classification pipeline. This stage was critical for quantifying the "convenience" of filtering; by comparing the accuracy and computational overhead of models trained on filtered versus raw data, we could determine if the reduction in data volume justifies the potential loss in signal or the added preprocessing latency.

4.2 Event Data and Datasets

Raw Data Acquisition

Two neuromorphic datasets were used in this project:

1. DSEC: Primarily used for testing and validating filtering techniques.
2. DVS Gesture: Used for both filtering experiments and the downstream classification task.

Both datasets provide asynchronous event streams that capture brightness changes over time, detected by event-based sensors. Unlike traditional frame-based cameras that capture intensity at fixed intervals, these sensors record per-pixel intensity changes. Each event e_i is defined by a tuple:

$$e_i = (x_i, y_i, t_i, p_i)$$

where (x_i, y_i) are the spatial coordinates of the pixel, t_i is the timestamp (typically in microseconds), and $p_i \in \{0, 1\}$ (or $\{-1, +1\}$) represents the polarity, indicating an increase or decrease in brightness.

Dataset Characteristics

The DVS Gesture dataset consists of recordings of 29 subjects performing 11 different hand and arm gestures under three different lighting conditions. The data is characterized by structured motion patterns suitable for categorical labeling. In contrast, the DSEC (Dataset for Driving Scenes with Event Cameras) provides high-resolution event data from a stereo event-camera setup mounted on a moving vehicle in various driving scenarios.

Initially, the datasets contained a total of 9,806,316 events for DVS Gesture and 213,025 events for DSEC. While both datasets were utilized to experiment with and refine the filtering algorithms, the downstream classification task was performed exclusively on the DVS Gesture dataset. The DSEC dataset was excluded from the classification phase because it is designed for regression-based tasks such as optical flow estimation, object detection in driving scenes, and stereo depth estimation, rather than discrete action or object classification. Its continuous, unstructured nature lacks the distinct class labels required to evaluate the impact of filtering on classification accuracy.

4.3 Filtering Techniques Implemented

In this section, we describe the various data reduction and filtering methodologies implemented to process the raw event streams. These techniques range from biologically-inspired models to statistical and heuristic-based approaches, each aim-

ing to minimize the event count while preserving the fundamental features required for classification. To evaluate their effectiveness, these methods are compared against random sampling baselines. The implemented filtering techniques are listed below:

- **OMS (Object Motion Sensitivity):** Motion-based saliency filtering.
- **Attention:** Learned importance weighting.
- **Adaptive Elbow:** Data-driven threshold selection.
- **Goal-Oriented:** Percentile-based event selection.
- **Mean–StdDev:** Statistical deviation filtering.
- **Global Saliency Crop:** Keeps most salient region only.
- **Denoising:** Removes isolated noisy events.
- **Random Crop:** Random data removal (baseline)

4.3.1 Hyperparameter Tuning and Optimization

For several of the implemented techniques, hyperparameter tuning was performed to enhance filtering efficacy and ensure the methods were properly calibrated to the specific dynamics of the DVS Gesture datasets. Due to project time constraints, optimization was conducted over a targeted range of values rather than an exhaustive grid search.

The tested percentages range from aggressive filtering (1–5%), where only highly salient regions are preserved, to mild filtering (30–50%), approaching the unfiltered case. This allows exploration of the accuracy–efficiency trade-off.
The final selection of optimal hyperparameters was determined by evaluating the accuracy of the downstream classification task. This ensured that the tuning process prioritized the preservation of task-relevant information over simple data reduction, identifying the "sweet spot" where noise is minimized without degrading the features necessary for high model performance.

4.3.2 The Event Reduction Ratio (ERR)

To quantitatively assess the impact of the filtering stage, we introduce the Event Reduction Ratio (ERR), defined as

$$\mathrm{ERR} = 1 - \frac{N_{\text{retained}}}{N_{\text{total}}},$$

where N_{retained} denotes the number of events that remain after applying the filtering, and N_{total} represents the total number of input events within the same time window.

The ERR measures the proportion of events suppressed by the filtering process. An ERR value close to 0 indicates minimal filtering (most events are retained), whereas a value approaching 1 corresponds to strong suppression of activity. This metric, therefore, provides a direct indication of how aggressively background motion and noise are reduced.

Importantly, ERR does not evaluate detection accuracy by itself; rather, it quantifies the level of data compression achieved prior to the downstream task. When interpreted together with task performance, ERR allows us to analyze the trade-off between event reduction and motion information preservation. A desirable operating point corresponds to a high ERR while maintaining stable or improved performance in the subsequent processing stage.

4.3.3 OMS: Motion-Based Saliency Filtering

To reduce background activity induced by camera motion and retain independently moving regions, we employ the Object Motion Sensitivity (OMS) model as a preprocessing stage. The method follows a center-surround architecture in which an accumulated event window is processed through two Gaussian convolutional pathways (center and surround), implemented using single-layer convolutional networks with Leaky Integrate-and-Fire (LIF) neurons. The OMS response is computed as the difference between the center and surround activations, thereby enhancing local motion contrast while suppressing coherent global motion.

The Center-Surround logic utilizes two distinct Gaussian kernels to process event-based data: a narrow center kernel that captures local activity and a wider surround kernel that captures the broader context. By subtracting the surround activity from the center, the system performs a spatial high-pass filter. In the context of OMS, if both the center and surround experience similar motion (global motion or "egomotion"), their signals largely cancel out. However, if an object moves differently than its background, a discrepancy occurs between the center and surround, resulting in a high saliency value. This allows the algorithm to suppress background noise caused by sensor movement while highlighting independent moving objects.

A saliency threshold is applied to determine which motion responses are consid-

ered significant. Only pixels whose motion intensity exceeds a predefined threshold are retained in the OMS map and used to filter incoming events. The threshold therefore controls the selectivity of the motion detection stage: lower values retain more events but may include noise, while higher values increase suppression but risk discarding relevant motion cues.

To identify the most suitable operating point, we performed hyperparameter tuning over some threshold values evaluating each configuration within the downstream task. The selected threshold corresponds to the value that yielded the best accuracy in the classification phase.

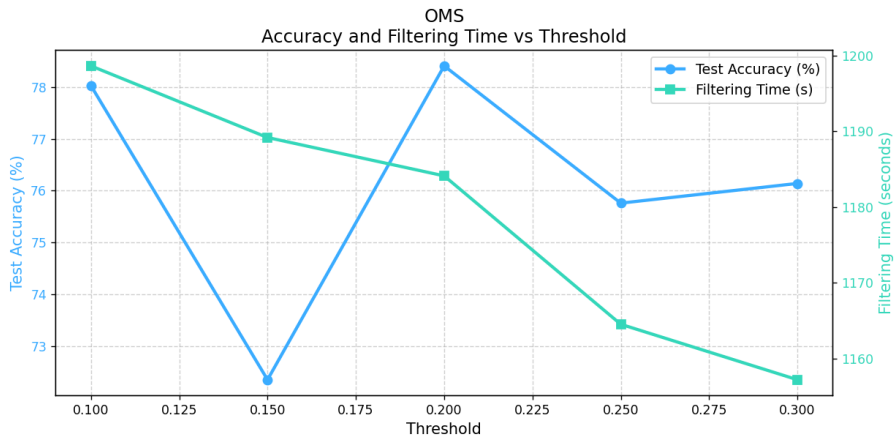

Figure 1: OMS hyperparameter comparison.

The threshold value chosen after hyperparameter tuning is the one that implied the highest accuracy, which in this case is 0.2.

4.3.4 Attention-Based Saliency Filtering

In addition to motion-based filtering, we implement an attention-driven saliency module to highlight structurally relevant regions in the event stream. The attention mechanism is based on a multi-orientation, multi-scale architecture that models border ownership and perceptual grouping through spiking convolutional layers.

Architecture Overview The attention module processes the accumulated positive and negative event maps as a two-channel input tensor. The core component is a three-level spatial pyramid, where each level operates at a progressively reduced spatial resolution. At every level, the input is processed by an *AttentionModuleLevel*, which consists of two main stages:

- **Border Ownership Estimation:** Orientation-selective filters based on von Mises distributions are applied through convolutional layers followed by Leaky

Integrate-and-Fire (LIF) neurons. For each orientation, responses are computed for both polarities and combined using inhibitory interactions. This stage enhances oriented contrast and encodes local border ownership cues.

- **Grouping Mechanism:** The border responses are further processed through grouped convolutions to promote spatial consistency and perceptual grouping. A winner-take-all strategy across orientations is applied, and inhibitory modulation controls competition between opposite border assignments. The resulting grouped activity forms a saliency representation at the current pyramid level.

The outputs of all pyramid levels are rescaled to a common resolution and summed to produce the final saliency map. The map is normalized to the range $[0, 1]$, providing a continuous representation of spatial attention strength.

Event-Level Filtering The normalized saliency map is used to filter events spatially. In contrast to OMS, where the threshold is manually tuned, the attention-based filtering employs an adaptive threshold defined as the mean saliency value of the map:

$$\theta = \text{mean}(S).$$

Events whose spatial coordinates satisfy $S(x, y) \geq \theta$ are retained, while the others are suppressed. This strategy dynamically adapts the selectivity of the filter to the overall saliency distribution of each frame.

As with OMS, the filtering strength is quantified using the Event Reduction Ratio (ERR). For this filtering technique, no hyperparameter tuning was performed.

4.3.5 Adaptive Elbow OMS Filtering

The Adaptive Elbow OMS Filtering technique is an advanced event-based data reduction method that builds upon the Object Motion Sensitivity (OMS) saliency map. While standard OMS filtering typically relies on a fixed global threshold to distinguish between relevant motion and background noise, this approach introduces an automated thresholding mechanism based on the geometric properties of the saliency distribution.

The core idea is to treat the selection of the filtering threshold as an optimization problem. In event-based vision, the saliency map (OMS map) often contains a small number of high-intensity pixels (representing significant motion) and a large number of low-intensity pixels (representing noise or static background). By analyzing the

cumulative distribution of these values, we can identify an "elbow" point, the point of maximum curvature, which naturally separates the signal from the noise.

Mathematical Formulation The filter identifies the optimal threshold by analyzing the sorted vector of non-zero saliency values, V, which are extracted from the OMS map and ordered from smallest to largest. We compute the normalized cumulative distribution C, representing the cumulative sum of these intensities, and a linear diagonal D. The curves are represented in a normalized space where the x-axis corresponds to the pixel index and the y-axis corresponds to the cumulative intensity, both scaled between 0 and 1. The threshold index k is selected by maximizing the distance between the distribution C and the diagonal D:

$$k = \arg\max_i |C_i - D_i| \tag{1}$$

The resulting threshold $T = V_k$ is used to generate a binary spatial mask $M(x, y)$.

Filtering Logic An event $e_i = (x_i, y_i, t_i, p_i)$ is retained in the filtered dataset if and only if:

$$M(x_i, y_i) = 1 \quad \text{where} \quad M(x, y) = \begin{cases} 1 & \text{if } OMS(x, y) \geq T \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

This ensures that the data reduction is tailored to the specific contrast and motion intensity of the current scene, optimizing the trade-off between data sparsity and information retention for downstream classification tasks.

4.3.6 Goal-Oriented OMS Filtering

This filtering technique implements a top-down filtering strategy, where the data reduction intensity is governed by a user-defined parameter. Unlike the Adaptive Elbow method, which seeks a statistical optimum, this approach enforces a specific retention rate, allowing the user to explicitly control the trade-off between event density and computational load for downstream tasks.

Percentile-Based Thresholding The core mechanism of this filter is the calculation of a threshold based on a desired percentage of data retention. Given a target parameter $keep_percent \in [0, 100]$, the algorithm determines the corresponding threshold value T from the Object Motion Sensitivity (OMS) map distribution.

Mathematically, if S is the set of all intensity values in the OMS map, the threshold T is defined as the $(100 - keep_percent)$-th percentile:

$$T = P_{100-keep_percent}(S) \tag{3}$$

This ensures that approximately $keep_percent$ of the spatial pixels in the OMS map are considered active.

Filtering Logic Starting from the OMS saliency map, a binary spatial mask M is generated where $M(y, x) = 1$ if $OMS(y, x) \geq T$, and 0 otherwise. Afterwards, each individual event $e_i = (x_i, y_i, t_i, p_i)$ is evaluated against the mask. An event is appended to the filtered stream if its spatial coordinates correspond to a mask location where $M(y, x) = 1$. The OMS saliency map is therefore used solely to generate the binary mask, which is subsequently applied to the raw event data. Finally, the filter computes the Event Reduction Rate (ERR), representing the fraction of the original data that has been discarded.

This filtering technique is particularly useful for establishing a baseline in classification experiments, as it allows for testing the impact of fixed data reduction levels (e.g., keeping only the top 10% or 20% of salient data) on accuracy. For this technique, the value of the $keep_percent$ was subject to hyperparameter tuning to find the value that maximizes the accuracy in the downstream task. In this case, the value chosen is 30.

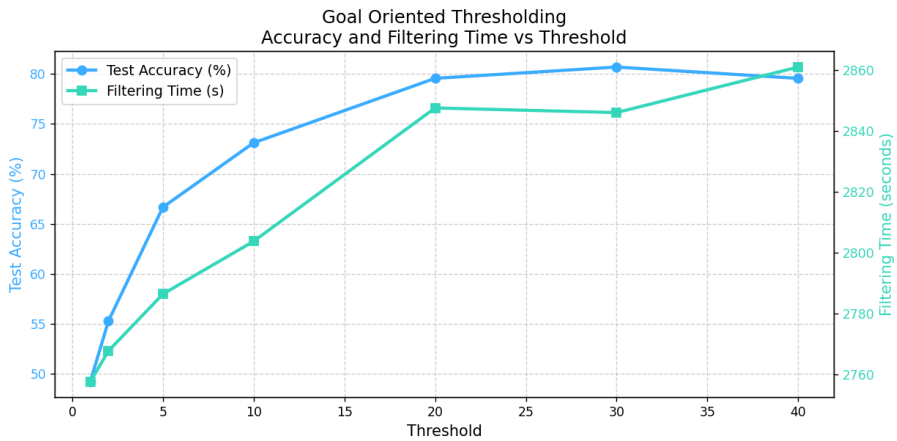

Figure 2: Goal Oriented Thresholding hyperparameter comparison.

4.3.7 Mean and Standard Deviation Thresholding

The Mean and Standard Deviation Thresholding filtering implements a statistical approach to event reduction still using the OMS Saliency map to generate the mask.

Statistical Threshold Derivation This technique assumes that lower-intensity values in the OMS map represent background noise or insignificant motion. To separate these from significant events, the algorithm calculates the mean (μ) and standard deviation (σ) of the entire flattened saliency map. The filtering threshold Θ is then defined as:

$$\Theta = \mu + k \cdot \sigma \tag{4}$$

where k is a user-defined scaling factor (the k-sigma multiplier). This approach is robust to varying global intensity levels, as the threshold shifts dynamically based on the overall statistical distribution of the map.

Binary Mask Application: A mask M is generated where:

$$M(y, x) = \begin{cases} 1 & \text{if } OMS(y, x) > \Theta \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

Afterwards, the algorithm extracts the valid subset of events where the mask value is 1. This preserves the original temporal and polarity information for all salient events while discarding those in suppressed regions.

As the k-sigma multiplier is user-defined, parameter tuing was used to determine the best value to assign, which in this case was 0.75.

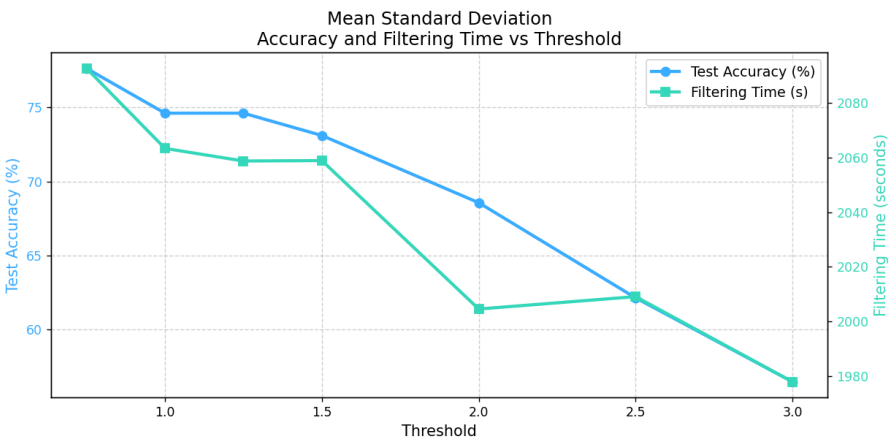

Figure 3: Mean and Standard Deviation Thresholding parameter comparison.

4.3.8 Global Saliency-Based Cropping

This filtering methodology introduces a spatial-reduction technique that goes beyond pixel-wise filtering by performing region-of-interest (ROI) extraction. Instead of only suppressing individual noise events, this method identifies the most salient spatial window within the event stream and crops the entire data volume to those boundaries.

Thresholding Strategies and Parameter Tuning The technique is designed with two distinct modalities to determine the threshold Θ, controlled by the `use_percentile` flag. This flexibility allows the user to tune the sensitivity of the salient region detection based on the specific characteristics of the dataset:

- **Percentile-Based Detection (`use_percentile=True`):** This adaptive strategy uses the `percentile` parameter (e.g., 90th percentile). It calculates a threshold Θ such that only the top $n\%$ of salient pixels are considered. This is highly effective for scenes with varying lighting or contrast, as it ensures a consistent proportion of the image is considered "active" regardless of absolute intensity values.
- **Fixed Threshold Detection (`use_percentile=False`):** This strategy uses a static Θ value (e.g., 0.4). It is preferred for controlled environments where the saliency values of the objects of interest are known and stable, providing a deterministic boundary for the cropping window.

Saliency-Driven Bounding Box The spatial region of interest is defined by generating a binary mask from the normalized Object Motion Sensitivity (OMS) map. Given the threshold θ the saliency mask is computed as:

$$M(y,x) = \begin{cases} 1 & \text{if } \mathrm{OMS}_{norm}(y,x) \geq \theta \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

Let $\mathcal{S}$ be the set of coordinates (x_{sal}, y_{sal}) where the mask is active:

$$\mathcal{S} = \{(x,y) \mid M(y,x) = 1\} \tag{7}$$

The boundaries of the spatial crop are then determined by calculating the extremes of the coordinates in $\mathcal{S}$:

$$x_{min}, x_{max} = \min(x_{sal}), \max(x_{sal}) \tag{8}$$

$$y_{min}, y_{max} = \min(y_{sal}), \max(y_{sal}) \tag{9}$$

Finally, an event $e_i = (x_i, y_i, t_i, p_i)$ is retained in the filtered stream if its spatial coordinates satisfy the bounding box constraints:

$$(x_{min} \leq x_i \leq x_{max}) \wedge (y_{min} \leq y_i \leq y_{max}) \tag{10}$$

The Event Reduction Rate (ERR) in this context represents the spatial efficiency of the crop, measuring how many events resided outside the primary area of motion.

Hyperparameter tuning was applied to both `percentile` and `threshold`; as a result the values that gave the highest accuracy were `percentile=85` and `threshold=0.15`.

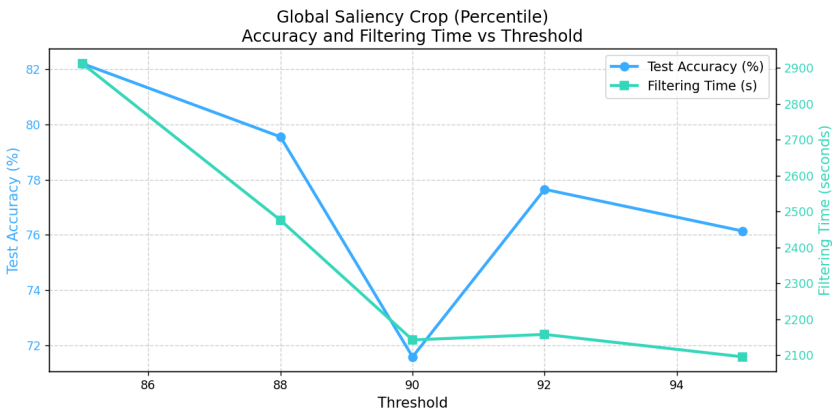

Figure 4: Global Saliency-Based Cropping `use_percentile=True` comparison.

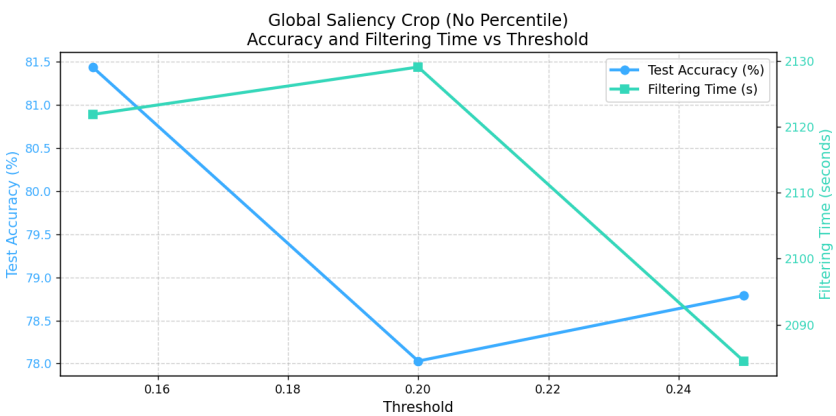

Figure 5: Global Saliency-Based Cropping `use_percentile=False` comparison.

4.3.9 Denoising

The `Denoise` method implements a standard event-based noise reduction filter, often utilized as a pre-processing step to eliminate isolated events that do not contribute to meaningful structural motion. This implementation is based on the background activity filter logic found in the `Tonic` library.

Connectivity-Based Filtering Logic The fundamental principle of this filter is that a valid event caused by real-world motion should be spatio-temporally correlated with other events. An event $e_i = (x_i, y_i, t_i, p_i)$ is considered "noise" and subsequently dropped if it is not sufficiently connected to its spatial neighborhood within a specific time window.

The algorithm maintains a 2D memory map, $M(x, y)$, which stores the timestamp of the last event processed at each pixel location, plus a predefined `filter_time` (Δt). For each incoming event at (x, y) and time t, the filter checks the 4-connectivity neighborhood:

$$\text{Keep } e_i \text{ if } \exists (x', y') \in \text{Neighborhood}(x, y) \text{ s.t. } M(x', y') > t \tag{11}$$

where the neighborhood includes pixels $\{(x-1, y), (x+1, y), (x, y-1), (x, y+1)\}$. If no neighboring pixel has recorded an event within the $[t - \Delta t, t]$ interval, the event is discarded as background noise.

The efficacy of the denoising process is quantified by the Event Reduction Rate (ERR)

4.3.10 Random Crop Filtering

The Random Crop Filtering method serves as a baseline data reduction technique. Unlike saliency-based methods that intelligently select regions of interest, this approach implements a stochastic spatial filter. It is primarily used to evaluate how a non-informed, blind reduction of the spatial field affects downstream classification accuracy compared to task-specific filtering.

Stochastic Spatial Reduction The core mechanism involves selecting a sub-window (crop) of a fixed size from the original sensor dimensions at a random location. Given a source sensor size (W, H) and a target crop size (W', H'), the algorithm determines the top-left coordinate (x_{start}, y_{start}) of the crop using a uni-

form distribution:

$$x_{start} \sim \mathcal{U}(0, W - W') \quad , \quad y_{start} \sim \mathcal{U}(0, H - H') \tag{12}$$

The events are then filtered based on their spatial coordinates. An event $e_i = (x_i, y_i, t_i, p_i)$ is retained if and only if:

$$x_{start} \leq x_i < x_{start} + W' \quad \text{and} \quad y_{start} \leq y_i < y_{start} + H' \tag{13}$$

Data Reduction Metrics The impact of the random crop is quantified by the Event Reduction Rate (ERR). Because the crop is placed randomly, the ERR can vary significantly between runs depending on whether the moving object in the scene was partially or fully captured within the random window.

Filtering Comparison on a dataset sample

The qualitative impact of the various filtering strategies is illustrated in Figure 6, which showcases the spatial transformation of a representative gesture sample from its raw state to its filtered representation. The *Event Map* serves as the unfiltered reference, characterized by a dense accumulation of asynchronous events and background activity.

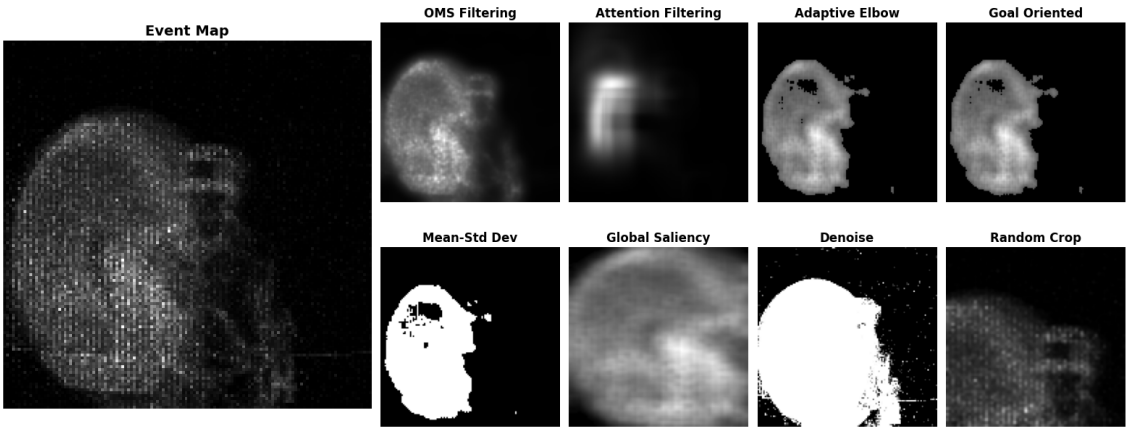

Figure 6: Visual comparison of the proposed filtering techniques on a single event-based sample from the DVS Gesture dataset.

4.4 Downstream Task: Classification Setup

To evaluate the impact of the proposed filtering techniques on data utility, a classification pipeline was implemented using a deep Convolutional Neural Network (CNN). The objective is to determine whether the reduction in event count (data sparsity) preserves or enhances the accuracy of gesture recognition compared to raw data.

4.4.1 Data Representation and Converters

Since standard CNNs operate on dense tensor grids, the asynchronous event stream must be converted into a frame-based representation. Four distinct conversion strategies were implemented to explore different facets of the event data:

- **Event Frame Converter:** Accumulates events into a single 2-channel frame (one per polarity), providing a high-speed representation but losing temporal evolution.
- **Stacked Frame Converter:** Subdivides the event window into N temporal bins, creating a $2N$-channel tensor that preserves the sequence of motion.
- **Time Surface Converter:** Encodes temporal decay based on the last event timestamp at each pixel using an exponential kernel $e^{-\Delta t/\tau}$.
- **Voxel Grid Converter (Selected):** Polarity is encoded as ± 1 and accumulated within each temporal bin B. This was selected as the primary representation using $B = 5$ bins and a 128×128 spatial resolution.

4.4.2 Model Architecture: DVSGestureCNN

The classification model is a Residual CNN designed to process the multi-channel voxel grids. The architecture consists of:

- **Initial Feature Extraction:** A 7×7 convolutional layer with stride 2 and Max-Pooling to reduce spatial dimensions while expanding channels to 64.
- **Residual Stages:** Three sequential layers composed of `ResidualBlocks`. Each block utilizes skip connections, Batch Normalization, and ReLU activation to mitigate the vanishing gradient problem. The channel depth increases from 64 to 256.
- **Classification Head:** Global Average Pooling (GAP) followed by a Dropout layer (0.5) and a fully connected layer mapping features to the 11 gesture classes of the DVSGesture dataset.

4.4.3 Training Methodology and Workflow

The experimental workflow follows a decoupled "Filter-then-Classify" approach to ensure consistency across benchmarks:

1. **Offline Filtering and Persistence:** Each filtering technique (OMS, Adaptive Elbow, Denoise, etc.) is applied to the raw DVSGesture dataset. The resulting event streams are saved locally as `.npy` files.

2. **Dataset Retrieval:** A custom `FilteredNPYDataset` wrapper is used to load these pre-computed datasets, ensuring the classifier sees exactly the same filtered data for every training run.

3. **Evaluation Metrics:** The primary metric is **Top-1 Accuracy** on the test set. Additionally, we monitor **Sparsity** (percentage of zero-elements in the representation) and **Training Time** to quantify the computational gain achieved by data reduction.

4.4.4 Hyperparameters Summary

Hyperparameter	**Value**
Input Dimensions	$5 \times 128 \times 128$ (Voxel Grid)
Optimizer	Adam ($lr = 1e-3$)
Loss Function	Cross Entropy
Batch Size	32
Epochs	20
Dropout Rate	0.5
Validation Split	20% of Training Set

Table 2: Classification model and training hyperparameters.

4.5 Experimental Environment

To provide a consistent baseline for the computational timing results, all filtering and classification tasks were executed on a dedicated workstation. The processing times reported in the results section, particularly the filtering latency which represents the primary computational bottleneck, are dependent on the following hardware and software configuration:

- **Hardware Specifications:**
  - **CPU:** Intel Core i9-10885H vPRO

  - **GPU:** NVIDIA Quadro RTX 5000
  - **RAM:** 64 GB

- **Software Environment:**
  - **Operating System:** Ubuntu 20.04 LTS
  - **Programming Language:** Python 3.11.2

5 Results

This section presents the quantitative evaluation of all filtering techniques applied to the DVS Gesture dataset, followed by a detailed performance analysis and discussion of trade-offs and limitations.

5.1 Experimental Setup

To assess the impact of event filtering on downstream classification, each filtering technique was applied to the DVS Gesture dataset prior to training and evaluating the same CNN architecture under identical conditions. Dataset splits, network architecture, optimizer configuration, and hyperparameters were kept constant across all experiments. The only varying component was the filtering strategy.

For reference, training the CNN directly on raw (unfiltered) event data achieved:

- Test Accuracy: 81.44%
- Training + Evaluation Time: 14.16 seconds

This baseline serves as the primary comparison point for all subsequent evaluations.

5.2 Quantitative Results

Data Reduction and Accuracy

Table 3: Data Reduction and Test Accuracy

Filtering Technique	**Data Removed (%)**	**Test Accuracy (%)**
OMS	27.42	79.17
Attention	27.73	72.35
Adaptive Elbow	80.53	71.59
Goal-Oriented	15.20	80.68
Mean–StdDev	28.03	77.27
Global Saliency Crop	9.13	81.44
Denoising	36.11	83.71
Random Crop	59.36	43.56

Computational Time

Table 4: Computational Time of Filtering Techniques

Filtering Technique	Filtering Time (s)	Training + Evaluation Time (s)
OMS	1166.01	12.44
Attention	2374.94	12.63
Adaptive Elbow	2194.77	12.36
Goal-Oriented	2849.37	12.50
Mean-StdDev	2084.99	12.55
Global Saliency Crop	2115.70	12.40
Denoising	2458.87	12.63
Random Crop	29.93	12.32

5.3 Accuracy vs. Data Reduction

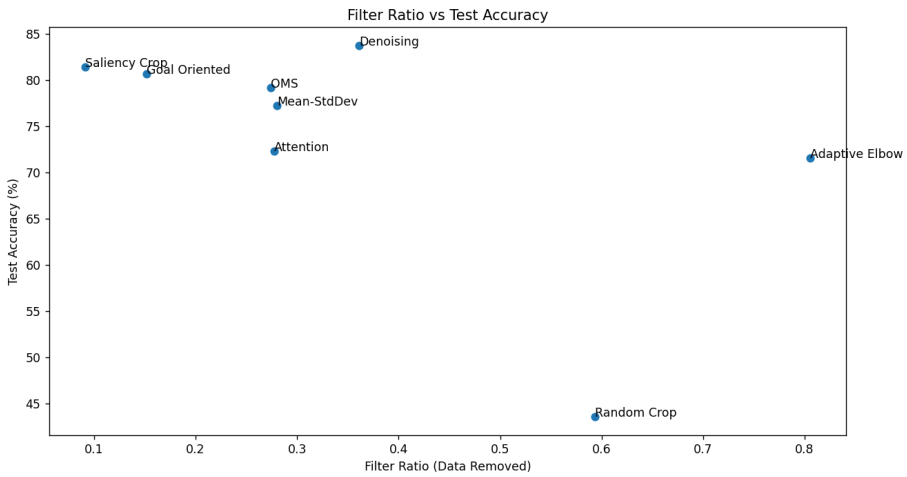

Figure 7: Test accuracy vs. data removed.

Figure 7 illustrates the relationship between event reduction and classification accuracy. Moderate filtering strategies such as Denoising and Goal-Oriented Thresholding maintain high accuracy while removing a meaningful proportion of events. In contrast, aggressive reduction (Adaptive Elbow) significantly decreases event density but leads to performance degradation, demonstrating that excessive filtering may discard discriminative information.

The Random Crop baseline further confirms that non-structured event removal severely harms performance.

5.4 Computational Efficiency

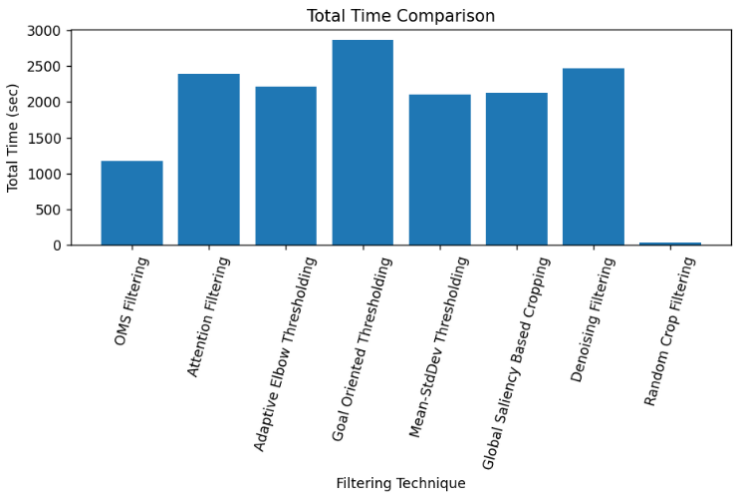

Figure 8: Total processing time across filtering techniques.

Figure 8 shows that filtering time dominates the overall processing pipeline. While CNN training and evaluation remain stable at approximately 12.5 seconds, filtering time varies substantially, ranging from 29 seconds (Random Crop) to nearly 2800 seconds (Goal-Oriented). This confirms that computational cost is primarily constrained by the filtering stage rather than classification.

5.5 Comparative Accuracy Analysis

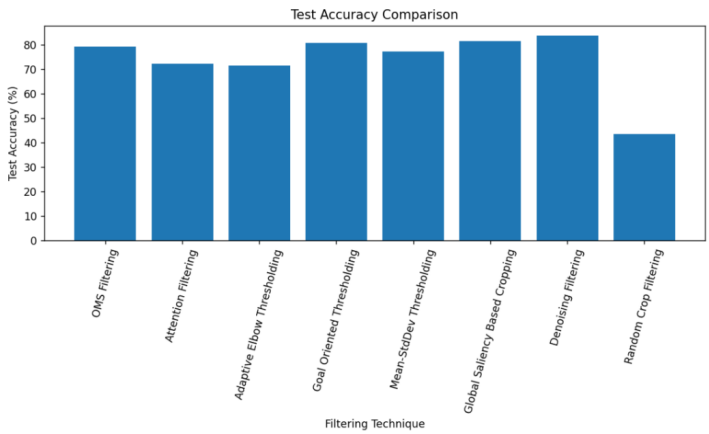

Figure 9: Comparison of test accuracy across filtering techniques.

As shown in Figure 9, Denoising achieves the highest test accuracy (83.71%), surpassing the raw baseline. Global Saliency Crop matches the baseline performance

(81.44%), while Goal-Oriented Thresholding also exceeds 80%.

Structured filtering methods consistently outperform naive filtering (Random Crop), demonstrating that intelligent event selection enhances discriminative feature learning.

5.6 Performance Observations

Several important trends emerge:

- All structured filtering techniques (except Random Crop) maintain competitive performance relative to the raw baseline.
- Denoising provides the highest accuracy while removing 36.11% of events.
- Adaptive Elbow removes the largest proportion of data (80.53%) but reduces accuracy significantly.
- Classification time remains nearly constant across experiments, indicating that model complexity is not the computational bottleneck.

These results highlight a fundamental trade-off between data reduction and classification performance.

5.7 Trade-Off Analysis

High-Accuracy Tier

Denoising, Global Saliency Crop, and Goal-Oriented Thresholding form a high-accuracy tier ($\geq$ 80%). These methods preserve or enhance discriminative information while moderately reducing event density. However, they incur substantial computational overhead, with filtering times exceeding 2000 seconds.

Balanced Performance

OMS Filtering provides a balanced compromise:

- 79.17% accuracy
- 27.42% data reduction
- Lower filtering time compared to most other structured methods

This makes OMS a competitive middle-ground solution.

Aggressive Reduction

Adaptive Elbow removes over 80% of events but reduces accuracy to 71.59%, indicating that excessive filtering risks discarding informative events necessary for classification.

Ineffective Baseline

Random Crop demonstrates that naive event removal is detrimental. Although extremely fast, the dramatic drop in accuracy confirms that filtering must respect the spatial-temporal structure of event streams.

5.8 Limitations

Despite the insights provided, several limitations must be acknowledged:

- **Single Dataset:** Evaluation was conducted only on DVS Gesture. Results may differ on other neuromorphic datasets.
- **Single CNN Architecture:** Only one classifier architecture was evaluated. Alternative models may respond differently to filtering strategies.
- **Parameter Sensitivity:** Some filtering methods required manual tuning, which may affect reproducibility.

6 Conclusions

This project investigated the impact of various saliency-based filtering techniques on the downstream classification of event-based data using the DVS Gesture dataset. By comparing biologically-inspired models, statistical thresholding, and stochastic baselines, the quantitative relationship between data reduction and task performance was established. Based on the experimental results, the following key conclusions are drawn:

- **Filtering Performance vs. Baseline:** Most structured filtering techniques, while reducing data volume, did not surpass the raw data baseline accuracy of 81.44%. Denoising Filtering was the only method to significantly outperform the baseline, reaching 83.71% accuracy. Global Saliency Based Cropping matched the baseline accuracy at 81.44%, while other methods like OMS (79.17%) and Goal-Oriented Thresholding (80.68%) resulted in slight performance decreases.

- **Optimal Accuracy and Denoising:** Denoising Filtering emerged as the most effective method for enhancing model performance, achieving the peak accuracy of 83.71% while removing 36.11% of the total events. This suggests that removing isolated, non-correlated events is more beneficial for classification than more aggressive saliency-based suppression.

- **Impact of Aggressive Data Reduction:** Adaptive Elbow Thresholding proved to be the most aggressive technique, removing 80.53% of the event volume. However, this high reduction rate led to a significant drop in accuracy to 71.59%, indicating that excessive filtering risks discarding critical semantic information necessary for gesture recognition.

- **Necessity of Structured Filtering:** The failure of the Random Crop Filtering baseline, which resulted in the lowest accuracy of 43.56%, confirms that data reduction must be guided by the underlying spatial and temporal structure of the event stream.

- **Computational Bottleneck:** While the classification time (training and evaluation) remained nearly constant at approximately 12.5 seconds, the total processing pipeline was heavily dominated by the filtering stage, with times often exceeding 2,000 seconds. This highlights a critical need for algorithmic optimization to make these preprocessing steps viable for real-time neuromorphic applications.

In conclusion, while raw event data provides a strong baseline, specific filtering like denoising can offer measurable improvements in classification accuracy. However, the substantial computational time required for filtering presents a major bottleneck. For event-based systems to remain efficient, the choice of filter must balance not only the reduction of data and task performance but also the processing latency added to the pipeline.

6.1 Future Work

Based on the findings of this project, several avenues for future research can be explored. First, it could be interesting to investigate algorithmic optimizations techniques for real-time processing, as the current filtering stage represents a significant computational bottleneck that dominates the total processing pipeline. Future efforts should focus on optimizing these filters for real-time execution using C++ or specialized neuromorphic hardware to align the preprocessing latency with the high-speed requirements of event-based sensors. Second, working on evaluation across diverse datasets and architectures is necessary to ensure the generalizability of these results. Since this study focused primarily on the DVS Gesture dataset and a specific CNN architecture, testing these filtering strategies on regression-based datasets like DSEC or alternative Spiking Neural Network (SNN) models would provide a more comprehensive understanding of their impact on various downstream tasks.

7 References

References

[1] Hugo Bulzomi et al. "Object Detection for Embedded Systems Using Tiny Spiking Neural Networks: Filtering Noise Through Visual Attention". In: *International Conference on Machine Vision and Applications (MVA)*. Vol. Proceedings of the 2023 18th International Conference on Machine Vision and Applications (MVA). Hamamatsu, Japan, July 2023. URL: https://hal.science/hal-04183160.

[2] Giulia D'Angelo et al. "Wandering around: A bioinspired approach to visual attention through object motion sensitivity". In: *arXiv preprint arXiv:2502.06747* (2025). URL: https://arxiv.org/abs/2502.06747.

[3] Amélie Gruel et al. "Neuromorphic foveation applied to semantic segmentation". In: *NeuroVision: What can computer vision learn from visual neuroscience? A CVPR 2022 Workshop*. This work was supported by the European Union's ERA-NET CHIST-ERA 2018 research and innovation programme under grant agreement ANR-19-CHR3-0008.The authors are grateful to the OPAL infrastructure from Université Côte d'Azur for providing resources and support. New Orleans, United States, June 2022. URL: https://hal.science/hal-03760724.